Motor de búsqueda de funciones similares vía AST

Trabajo Práctico Grupal · Lenguajes Formales y Autómatas 2026 · UCSE
Grupo 7: Gauchat, Mallet y Poggi

Objetivo y decisiones de diseño

Implementamos una herramienta de consola que busca funciones estructuralmente similares en una carpeta local con archivos Python. El alcance previsto es moderado —decenas de archivos— y la unidad de comparación es cada función, no el archivo completo. Organizamos el flujo en cuatro módulos: extracción, normalización y serialización, cálculo de similitud y búsqueda con presentación de resultados.

El extractor recorre recursivamente los archivos .py, los abre respetando la codificación declarada y construye sus árboles con *ast.parse()*. Registra nombre, ruta relativa y línea de cada *FunctionDef*, incluidos métodos y funciones anidadas. Si un archivo no se puede leer o parsear, lo omite y comunica un aviso. No importamos ni ejecutamos los archivos examinados: trabajamos sobre su estructura sintáctica. Las funciones asíncronas no forman parte de los objetivos.

Para comparar estructuras sin depender de nombres propios, normalizamos una copia del AST. El nombre de la función raíz pasa a *FUNCION*; parámetros e identificadores reciben nombres *VAR1*, *VAR2*, etc., conservando la correspondencia que registra el recorrido. Las constantes pasan a *LIT*. Quitamos docstrings iniciales y comentarios de tipo, y conservamos operadores y estructura. Luego serializamos el árbol en pre-order: incluimos tipos de nodos, campos, valores y marcas de inicio y fin para distinguir listas, bloques y valores ausentes. Esta relación con la materia está en representar y recorrer la estructura sintáctica del programa mediante un AST; no implementamos un parser desde cero.

Métrica y comprobación

Elegimos Jaccard sobre conjuntos de trigramas consecutivos de tokens. Con n = 3, cada elemento compara una ventana local de tres tokens; el conjunto permite una implementación directa, aunque descarta cuántas veces aparece un mismo trigrama. Fijamos este tamaño y el umbral en 0,70 después de comprobar los ejemplos acordados. La similitud se calcula así:

J(A, B) = |A intersección B| / |A unión B|

A y B son los conjuntos de trigramas de cada función. El buscador ordena las funciones por score y muestra las primeras K; el umbral etiqueta una función como candidata cuando el valor sin redondear es al menos 0,70. La etiqueta no elimina resultados del ranking y no afirma que exista duplicación.

Los dos positivos definidos antes de medir —sumar/total y acumular/contar, clones con renombrado— obtuvieron 1,0000. Los dos negativos previstos —sumar/contar e identidad/acumular— obtuvieron 0,5620 y 0,4324, respectivamente. El umbral separa estos cuatro casos. En el ejemplo exploratorio, operaciones y operaciones_reordenadas obtuvieron 1,0000 pese a tener tokens distintos: su conjunto de trigramas coincide. Esto muestra que Jaccard pierde parte del orden global; un score perfecto no prueba igualdad de AST ni equivalencia de comportamiento. La suite de 64 pruebas pasó en Python 3.13.15 el 29/10/2026. Ese resultado comprueba los casos implementados, pero no mide la precisión general del detector.

Búsqueda, limitaciones y mejoras posibles

La interfaz selecciona el objetivo mediante ruta relativa y nombre; si el nombre se repite en el archivo, permite precisar la línea. Comparamos el objetivo con las demás funciones, ordenamos por score descendente y resolvemos empates por archivo y línea. El parámetro K limita cuántos resultados se muestran. Los avisos de archivos omitidos y los errores de entrada se comunican por consola. Así, el ranking sirve para revisar candidatos dentro de la carpeta elegida sin convertir el umbral en un filtro de resultados.

La comparación es estructural y aproximada. Reemplazar literales puede acercar funciones con valores y comportamientos distintos. El mapa de identificadores es único por árbol y no resuelve ámbitos ni sombreado; por eso puede confundir identificadores iguales en ámbitos distintos o no reconocer renombrados que dependen del ámbito. También conservamos nombres de atributos, módulos importados y etiquetas de argumentos por nombre, que pueden influir en el score. Como Jaccard opera sobre conjuntos, pierde frecuencia y parte del orden; no garantiza detectar cualquier reordenamiento ni transformaciones algorítmicas equivalentes. La calibración se limita a unos pocos ejemplos y el uso previsto a carpetas de tamaño moderado.

Como trabajo futuro, proponemos ampliar el corpus con más pares positivos y negativos; analizar los ámbitos al renombrar identificadores; y evaluar una política alternativa para normalizar literales o la distancia de edición normalizada permitida por la consigna. Habría que medir cada cambio antes de decidir si mejora la separación de los casos.

Fuentes: consigna oficial del Trabajo Práctico (UCSE, 2026); documentación oficial de *ast* de Python 3.13: https://docs.python.org/3.13/library/ast.html.